TRACE: A Graph-Constrained Transformer for Communication-Efficient Distributed Routing in LEO Constellations

Qiuchao Dai, Zhongyuan Jiang, Fanxuan Sun, Xinghua Li, *Member, IEEE*, and Jianfeng Ma, *Member, IEEE*

***Abstract*—Large-scale Low Earth Orbit (LEO) constellations often experience high node failure rates caused by dynamic environmental factors (random failures), such as satellite maneuvers, or by cyber or physical attacks on critical nodes (targeted attacks), which pose unique challenges for routing optimization. Traditional algorithms such as Dijkstra suffer from limited parallel scalability on GPUs due to irregular neighbor distributions, while deep learning methods lack generalization ability. To address these challenges, we propose TRACE (Topology-aware Routing via Adjacent-Constraint Encoding), a graph-constrained Transformer architecture equipped with a cascaded multi-head attention decoder for distributed dynamic routing in LEO domains. To improve algorithm throughput and resilience against random failures and targeted attacks, we further design NFD (Navigator–Follower Distillation), a self-distillation framework which enables each agent to learn routing policies from Monte Carlo episodes. Furthermore, a hierarchical distributed routing architecture is developed to extend the proposed method to multi-domain scenarios. Simulation results demonstrate that TRACE with NFD achieves near-optimal routing accuracy with controllable inter-domain errors, while significantly improving throughput compared with mainstream routing algorithms.**

***Index Terms*—LEO constellations, distributed optimization, dynamic networks, hierarchical routing, knowledge distillation.**

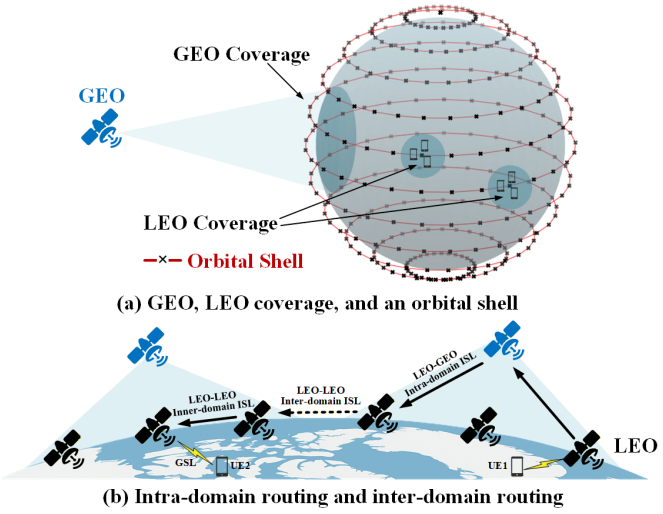


Fig. 1. GEO-LEO coverage and cross-domain communication. (a) In an orbital shell, LEO satellites covering UEs on the earth are also covered by GEO satellites. (b) GEO satellites naturally divide the LEO constellation into multiple domains, and UE1 have to transfer packets to UE2 across domains.

I. Introduction

A. Background and Motivation

LOW Earth Orbit satellite constellations are rapidly becoming a key part of global communication systems, offering wide coverage and low latency [1]. As shown in Figure 1, a Geostationary Earth Orbit (GEO) satellite offers excellent coverage at high altitudes and can access to LEO satellites through optical links [2]. In the GEO-LEO system, the coverage of each GEO satellite naturally divides the LEO constellation into multiple domains, where each domain operates as an independent routing area [2]. User equipments (UEs) transfer data to the corresponding LEO domain through Ground Satellite Links (GSLs), and the data is forwarded through intra-domain and inter-domain inter-satellite links (ISLs). Prior studies show that GEO is better employed as an extra traffic forwarder for intra-domain routing [3] due to its relatively lower link capacity [4], [5].

However, LEO constellations experience two primary types of node failures: random failures caused by space radiation, dynamic ISLs, and energy constraints, and targeted attacks on critical nodes [1]. These factors jointly result in highly dynamic network topologies. Furthermore, the expansion of constellation scale significantly complicates on-orbit maintenance, which exacerbates node failures. Consequently, routing within the network becomes much more challenging than in terrestrial systems.

Received 8 January 2026; revised 27 April 2026; accepted 27 May 2026. Date of publication 9 June 2026; date of current version 18 June 2026. This work was supported in part by the National Science and Technology Major Project of China (Grant No. 2025ZD1303100), Natural Science Basic Research Program of Shaanxi under Grant 2025JC-JCQN-085, and the Fundamental Research Funds for the Central Universities of China under Grant QTZX26140. The associate editor coordinating the review of this article and approving it for publication was J. Wang. *(Corresponding author: Zhongyuan Jiang.)*

Qiuchao Dai, Fanxuan Sun, Xinghua Li, and Jianfeng Ma are with the School of Cyber Engineering and the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an, Shaanxi 710071, China (e-mail: qiuchao@stu.xidian.edu.cn; fanxuansun@stu.xidian.edu.cn; xhli1@mail.xidian.edu.cn; jfma@mail.xidian.edu.cn).

Zhongyuan Jiang is with the School of Cyber Engineering and the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an, Shaanxi 710071, China, and also with Purple Mountain Laboratories, Nanjing, Jiangsu 211111, China (e-mail: zyjiang@xidian.edu.cn).

Digital Object Identifier 10.1109/TCOMM.2026.3701848

0090-6778 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

These challenges jointly give rise to three fundamental difficulties in routing: (i) frequent topology changes demand adaptive decision-making; (ii) dynamic ISLs result in shrinking link availability windows, thereby requiring more efficient routing computation; and (iii) larger constellation sizes call for scalable architectures.

Survey papers show that LEO networks must frequently reconfigure routing paths as topology changes [6]. Some research [7] have tried to solve the problem through RL (Reinforced Learning), but they also face the difficulty of excessively long model convergence time after topological changes.

Traditional routing methods such as Dijkstra's algorithm (and similar shortest-path approaches) assume relatively stable networks and uniform node degrees [8]. On modern GPU hardware, however, their performance is limited when neighbor sets vary widely. For instance, work on graph processing on GPUs reports that irregular workloads and control flow hamper effective parallelism [9].

Furthermore, centralized routing architectures cannot scale with the growth of large LEO constellations, leading to excessive signaling and poor adaptability [8]. To cope with high dynamic topologies and limited inter-satellite bandwidth, distributed routing architectures [7], [10] are increasingly adopted to enable local autonomy and cooperative optimization among satellites.

Based on these challenges and the limitations of existing methods, we conclude that routing in large-scale, dynamic LEO constellations requires three key advances.

- **Adaptive:** Routing algorithms must adapt to topology changes and achieve near-optimal performance comparable to traditional algorithms.
- **Efficient:** The algorithm must be efficient enough to take full advantage of many-core hardware without being bottlenecked by synchronization or imbalance.
- **Scalable:** The routing architecture must be scalable to large LEO constellations and remain highly stable.

B. Related Work

Existing routing approaches for LEO and dynamic networks can be broadly categorized based on how they address the three fundamental challenges identified in Section I, namely adaptivity, efficiency, and scalability.

Scalability-oriented approaches. To improve routing scalability and reliability in large-scale constellations, several studies focus on protocol-level and architectural designs. Dui et al. [11] and Lai et al. [12] investigate topology perturbations from the perspectives of attack strategies and congestion control, respectively, while Li et al. [2] and Du et al. [13] enhance stability through hierarchical routing domains and segmented resilient routing. These approaches aim to improve system-level scalability and fault tolerance by organizing large constellations into manageable structures. However, they primarily rely on protocol and control-plane mechanisms, with limited support for fast routing decision-making and algorithmic scalability under highly dynamic topologies.

Adaptivity-oriented approaches. With the rise of ubiquitous intelligence in 6G, learning-based methods are increasingly adopted to improve adaptivity in dynamic environments. Wu and Huang [14] employs reinforcement learning for dynamic ISL switching, while Zhou et al. [15] and Ran et al. [7] design distributed routing strategies using PPO, POMDP, and graph attention networks (GAT). Other works explore graph-based or optimization-driven methods, such as GCN-based routing [16], evolutionary multi-objective optimization [17], and continual deep reinforcement learning [18]. In addition, recent studies incorporate federated learning, generative AI, and mixture-of-experts models to enhance distributed intelligence and resource efficiency in NTN systems [19], [20], [21], [22], [23], [24], [25], [26], [27], [28]. These approaches improve adaptability to dynamic conditions through data-driven decision-making. However, most of them focus on learning frameworks or resource management rather than routing performance, and often lack systematic analysis of real-time decision-making, agent deployment, and scalability in large-scale LEO constellations.

Efficiency-oriented approaches. Classical shortest-path algorithms, such as Dijkstra and Bellman–Ford, provide efficient routing solutions under known topology conditions. Uditi et al. [29] and Jasika et al. [30] show that even with parallelization techniques such as OpenMP or OpenCL, these algorithms remain constrained by strong sequential dependencies. While they can achieve optimal routing decisions when complete and accurate topology information is available, their limited parallel scalability and reliance on global state make them less suitable for real-time routing in highly dynamic and large-scale LEO networks.

In summary, existing approaches exhibit complementary strengths but fail to simultaneously address the three key requirements. Protocol-level methods improve scalability but lack efficient decision-making; learning-based approaches enhance adaptivity but often suffer from high inference overhead or limited scalability; and classical algorithms provide efficiency under full information but are sensitive to incomplete or outdated topology states. These limitations motivate the need for a routing framework that achieves high throughput, scalability, and robustness under dynamic and resource-constrained LEO environments.

Compared with existing approaches, TRACE fundamentally differs by integrating topology-aware representation learning with a cascaded decision mechanism and a lightweight training framework, enabling both high-throughput inference and robust routing under constrained synchronization.

C. Our Method

These three requirements motivate us to propose TRACE (Topology-aware Routing via Adjacent-Constraint Encoding) and NFD (Navigator–Follower Distillation). To address the adaptive and efficient requirements, TRACE leverages a Transformer architecture with adjacency-constrained attention, which naturally supports parallel decision-making over irregular graph structures while preserving local connectivity constraints. This design enables scalable and generalizable

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

routing under dynamic LEO topologies. To enhance robustness under failures and obtain a better-performing model, NFD introduces a self-distillation mechanism that combines Monte Carlo exploration with policy learning, allowing the training process to capture the relative quality of candidate routing actions beyond ground-truth, thereby improving resilience to random failures and targeted attacks. We also design a hierarchical distributed routing architecture to scale from a single domain to multi-domain constellations. The results show that TRACE with NFD consistently achieves near-optimal routing accuracy, maintains control over inter-domain errors, and significantly improves throughput compared with mainstream routing solutions. The main contributions are as follows.

- **Generalizable:** We develop a graph-constrained Transformer for routing that embeds adjacency constraints directly into the attention mechanism, enabling robust and generalizable dynamic routing in LEO constellations.
- **High-throughput:** We design a distillation-based training mechanism (NFD) that allows agents to learn routing policies under failure scenarios through Monte Carlo exploration and feedback, which achieves higher throughput compared with mainstream algorithms.
- **Distributed:** We propose a hierarchical distributed routing framework to extend the approach from a single-domain case to multi-domain environments and validate its performance through simulation.

The remainder of this paper is organized as follows: Section II presents the system model and problem formulation; Section III details the proposed TRACE and NFD methods; Section IV reports experimental results and analysis; finally, Section V concludes our work.

II. System Model and Problem Formulation

In this section, we first introduce the network topology and communication model under two types of node failures (i.e., random failures and targeted attacks), followed by a GEO optical link allocation scheme to enhance the network resilience. We then propose a hierarchical multi-domain distributed routing framework and formulate the corresponding problem. Finally, we give the detailed distributed routing workflow at each satellite node.

A. Network Topology and Communication Model

Assume that the LEO satellites $\mathcal{S} = \{1, 2, \ldots, S\}$ is divided by GEO into K domains, i.e., $\mathcal{K} = \{1, 2, \ldots, K\}$, as shown in Figure 1 (b). The intra-domain satellites form a classic mesh topology as shown in Figure 2. Then the k-th domain can be expressed as a graph $\mathcal{G}_k = (\mathcal{V}_k, \mathcal{E}_k)$, where $\mathcal{V}_k$ is the set of intra-domain satellites, $\mathcal{E}_k$ is the set of intra-domain ISLs. Above single domains, there is a graph $\mathcal{G}_{inter} = (\mathcal{K}, \mathcal{E}_{inter})$ describing the overall topology, where $\mathcal{E}_{inter}$ denotes the inter-domain ISLs.

For $i, j \in \mathcal{S}$, if there is an optical link (i.e., ISL) between i and j, the SNR (Single-to-Noise Ratio) is approximated by (1):

$$SNR_{i,j} = \frac{CP}{\rho B d_{i,j}^2}, \tag{1}$$

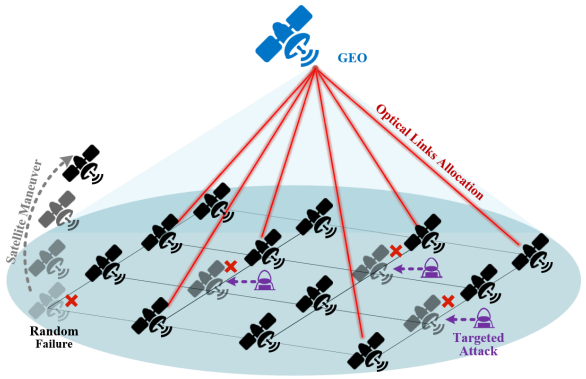


Fig. 2. GEO optical links allocation against targeted attacks and random failures.

where P is the transmission power, C is a constant related to channel gain, $d_{i,j} \sim N(d, \sigma_d^2)$ is the satellite distance, ρ is the noise density, and B is the bandwidth. The transmission delay $T_{i,j}^c$ and propagation delay $T_{i,j}^p$ can be obtained by (2):

$$T_{i,j}^c = \kappa / R_{i,j}, \tag{2a}$$
$$T_{i,j}^p = d_{i,j} / c, \tag{2b}$$

where κ is the size of the transmitted packet data, $R_{i,j} = Blog(1+SNR_{i,j})$ refers to the transmission rate, and c is the light speed.

Apart from the transmission and propagation delays, the queueing delay is also considered. Let $\mu_j = \sum_{i \in \mathcal{N}(j)} R_{j,i}$ denote the aggregate outgoing service rate of node j, where $\mathcal{N}(j)$ indicates the neighbors of node j. For tractability, we adopt a Markovian queueing approximation, which is widely used for communication-network delay modeling [31], [32]. Specifically, the traffic arrival rate is set as $\lambda_j = \eta \mu_j$, where $0 < \eta < 1$ is the traffic load factor and $\eta = 0.5$ is used in this paper.

Since the focus of this work is routing decision rather than fine-grained traffic modeling, the queue delay is modeled by a truncated exponential random variable:

$$Q_j = \min\{\tilde{Q}_j, Q_{\max}\}, \quad \tilde{Q}_j \sim \mathrm{Exp}(1/\bar{Q}_j), \tag{3}$$

where $Q_{\max}$ is the maximum queue capacity and $\bar{Q}_j$ denotes the average queue backlog. Then, the queueing delay at node j is given by (3)

$$T_j^q = Q_j / \mu_j. \tag{4}$$

This exponential assumption provides a lightweight baseline queueing model. Although more bursty traffic can be described by more complex models, such as ON/OFF or Markov-modulated processes, the proposed routing framework only requires the resulting link-delay matrix as input and can be extended by replacing Q_j with other traffic distributions.

The above described communication model represents the general case, while in certain special scenarios, nodes may face two kinds of potential failures. As shown in Figure 2, node failures may be caused by satellite maneuvers and targeted

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

attacks. Considering node failures, the delay $T_{i,j}$ for packet data transmission from i to j is given by (5):

$$T_{i,j} = \begin{cases} +\infty, & \text{node } j \text{ inaccessible} \\ T^c_{i,j} + T^p_{i,j} + T^q_j, & \text{otherwise.} \end{cases} \tag{5}$$

1) *Random Failure:* Satellites have to maneuver drastically to avoid possible collisions [2], which may lead to the interruption of ISLs. In this case, we assume that each satellite performs orbital maneuvers and fails with equal probability p_{rand}. The random node failure probability p^{rand}_j can be expressed as (6):

$$p^{rand}_j = p_{rand}, j \in \mathcal{S}. \tag{6}$$

2) *Targeted Attacks:* Attackers may adopt attacks like DDoS or Jamming to target critical satellite nodes, causing link disruption or severe degradation. In this case, we assume that the probability of a node being attacked is proportional to its impact on the algebraic connectivity. If we denote $alg(\cdot)$ as the algebraic connectivity and $\mathcal{G}^{-j}_k = (\mathcal{V}_k \setminus \{j\}, \mathcal{E}_k \setminus \{e^k_{ij} | i \in \mathcal{N}(j)\})$, the node failure probability p^{tar}_j under targeted attacks can be expressed as (7):

$$p^{tar}_j = \frac{alg(\mathcal{G}_k) - alg(\mathcal{G}^{-j}_k)}{\sum_{i \in \mathcal{V}_k}(alg(\mathcal{G}_k) - alg(\mathcal{G}^{-i}_k))}, j \in \mathcal{V}_k. \tag{7}$$

As is depicted in Figure 2, the GEO satellite is introduced to enhance the network resilience against node failures. By allocating optical links, GEO satellites relay traffic for LEO domains, providing additional connectivity. The routing decisions are performed by LEO satellites in a distributed manner based on local topology information, while GEO–LEO optical links are treated as optional relay paths when available.

However, the number of allocable links for GEO satellites is limited, and attackers can also attack GEO satellites through LEO nodes. Therefore, to reduce the workload of GEO satellites and maximize the connectivity of LEO domains, we adopt the allocation strategy proposed in [33]. The number of optical links $\mathcal{A}_k$ to be allocated to the k-th domain can be denoted by (8):

$$\mathcal{A}_k = \min\left\{ \left\lfloor A \cdot |\mathcal{V}_k|^{\frac{2}{3}} \right\rfloor, |\mathcal{V}_k| - 1 \right\}, \tag{8}$$

where A is a constant related to the average degree of the LEO network.

In this paper, we allocate GEO optical links in descending order of LEO node algebraic connectivity. After the allocation, $\mathcal{G}_k$ achieves improved resilience against node failures.

B. Multi-Domain Distributed Routing Problem Formulation

We propose a hierarchical distributed architecture for multi-domain routing, Figure 3 shows the overall framework. The distributedly deployed agents share the same model parameters with a prototype, and are responsible for both inter-domain routing and intra-domain routing. It is worth noting that the agents are only deployed to on-orbit Base Stations (BSs) or gateway satellites to save the computing power of LEO domains. Through hierarchical routing, the constellation routing among S nodes is reduced to K intra-domain routing and one inter-domain routing computation.

Given the constellation topology $\mathcal{G}_{inter}$, and a routing request $req = (src, dst, \kappa)$ from $src \in \mathcal{V}_{k_{src}}$ to $dst \in \mathcal{V}_{k_{dst}}$, we denote a valid inter-domain route path by (9):

$$\mathbf{p}^{inter}_{req} = [k_0, k_1, \ldots, k_t, \ldots, k_T],\ k_t \in \mathcal{K}, \tag{9}$$

where $k_0 = k_{src}$, $k_T = k_{dst}$, $T = |\mathbf{p}^{inter}_{req}|$, and $k_t \in \mathcal{N}(k_{t-1})$.

It is worth noting that the source node src_{k_t} of domain k_t is only related to $k_{t-1},\ 1 \le t \le T$, and the destination node dst_{k_t} is only related to $k_{t+1},\ 0 \le t \le T-1$. In addition, $src_{k_0} = src$ and $dst_{k_T} = dst$.

In real LEO constellations, domain k and $k+1$ may be connected by multiple ISLs. In other words, $\langle k, k+1 \rangle$ cannot be mapped one-to-one to a specific ISL. However, this can be reduced to the single-ISL case by introducing auxiliary domains. In particular, the i-th edge of $\langle k, k+1 \rangle$ can be converted into $\langle k, k^{aux}_i \rangle$ and $\langle k^{aux}_i, k+1 \rangle$, where k^{aux}_i is an empty domain, and any data entering k^{aux}_i will be immediately forwarded to domain $k+1$ without processing. For ease of performance evaluation, we assume that k^{aux}_i is not an empty domain; that is, all K domains crossed by a packet correspond to a real single-domain topology.

Similarly, given the topology $\mathcal{G}_{k_t}$ of the k_t-th domain we denote a valid intra-domain route path by (10):

$$\mathbf{p}^{k_t}_{req} = [v^{k_t}_0, v^{k_t}_1, \ldots, v^{k_t}_\tau, \ldots, v^{k_t}_{\mathcal{T}}],\ v^{k_t}_\tau \in \mathcal{V}_{k_t}, \tag{10}$$

where $v^{k_t}_0 = src_{k_t}$, $v^{k_t}_{\mathcal{T}} = dst_{k_t}$, $\mathcal{T} = |\mathbf{p}^{k_t}_{req}|$, and $v^{k_t}_\tau \in \mathcal{N}(v^{k_t}_{\tau-1})$. By combining (9) and (10), the overall route $\mathbf{p}_{req}$ can be obtained as (11):

$$\mathbf{p}_{req} = \mathbf{p}^{k_0}_{req} \oplus \mathbf{p}^{k_1}_{req} \ldots \oplus \mathbf{p}^{k_t}_{req} \ldots \oplus \mathbf{p}^{k_T}_{req},\ k_t \in \mathbf{p}^{inter}_{req}, \tag{11}$$

where $\oplus$ means the ordered concatenation of two sequences. Then the communication delay $\mathcal{D}$ is denoted by (12):

$$\mathcal{D}(\mathbf{p}_{req}) = \sum_{[i,j] \subseteq \mathbf{p}_{req}} T_{i,j}. \tag{12}$$

Actually, the routing process is a Markov Decision Process (MDP), where each hop is determined by the distributed agents. The agents achieve implicit consensus by sharing parameters, where π is the routing policy, θ and ϕ are the parameters for inter and intra domain routing respectively.

The probability of the action performed by the agent to choose k_t is only related to the input sate $s^{inter}_t = \langle k_{t-1}, k_{dst}, \mathcal{G}_{inter} \rangle$, and the inter-domain routing policy π_θ with parameters θ, i.e., $p(k_t | s^{inter}_t; \theta) = \pi^t_\theta(k_t)$. Similarly, we have $p(v^{k_t}_\tau | s^{intra}_\tau; \phi) = \pi^\tau_\phi(v^{k_t}_\tau)$. To ensure consistency, we define $\pi^0_\theta(k_0) = 1$ and $\pi^0_\phi(v^{k_t}_0) = 1$. Then, the inter and intra domain path probability can be written uniformly as (13).

$$\pi_{param}(\mathbf{p}|s) = \prod_{i=0}^{|\mathbf{p}|} \pi^i_{param}(\mathbf{p}^{(i)}), \tag{13}$$

where s is the initial state, $\mathbf{p}$ is the output path, $param$ is θ or ϕ, and $\mathbf{p}^{(i)}$ is the i-th node in the path.

Finally, we formulate the distributed routing problem as $\mathbf{P}$, where $\mathbb{E}[\cdot]$ represents the expectation under node failures, random routing requests, and different constellation topologies.

$$\mathbf{P}: \min_{\theta,\phi} \mathbb{E}[\mathcal{D}(\mathbf{p}_{req})]$$

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

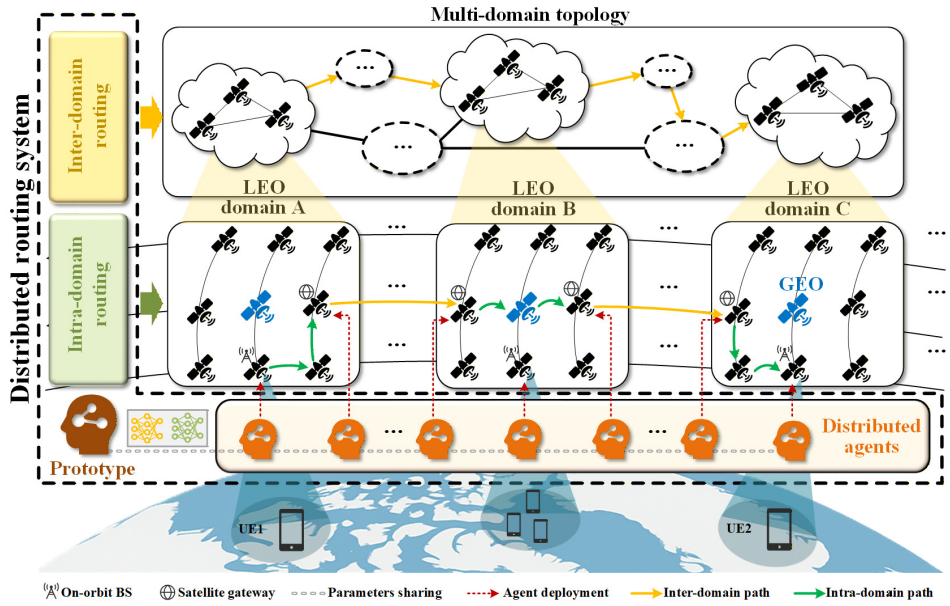


Fig. 3. Distributed routing framework. Distributed agents deployed to on-orbit BSs or gateway nodes, share the inter-domain and intra-domain routing parameters with the Prototype model. Packets from UE1 are routed through LEO domains A, B, and C to reach UE2.

$$\text{s.t. } |\mathbf{p}_{req} \cap \{dst\}| = 1, \tag{14a}$$

$$\mathcal{D}(\mathbf{p}_{req}) < +\infty, \tag{14b}$$

$$|\mathbf{p}_{req}^{k}| \leq H, \ \forall\, k \in \mathbf{p}_{req}^{inter}, \tag{14c}$$

$$\mathbf{p}_{req}^{(i)} \neq \mathbf{p}_{req}^{(j)}, \ \forall\, i \neq j. \tag{14d}$$

Among the constraints, (14a) guarantees that the packet can be transferred to the destination; (14b) ensures that the path contains no failed nodes and conforms to the graph constraints; (14c) defines the maximum hop limit H in a single domain; (14d) enforces a loop-free path.

C. Multi-Domain Distributed Routing Workflow

We next give the detailed workflow in each satellite in the proposed distributed routing framework. The overall workflow is divided into two planes, i.e., a decision plane and a forwarding plane. The decision plane operates through distributed agents, while the forwarding plane is executed by satellite routers. It is worth noting that each satellite is equipped with a router, but not all of them have agents deployed. Routing decisions are triggered only at nodes equipped with agents, while other nodes follow forwarding instructions provided by previous decisions.

1) *Decision plane:* User packets enter LEO domains through GSLs or inter-domain ISLs, where the entry points are on-orbit BSs or satellite gateways deployed with agents. The agent will first compute the inter-domain route to deliver the packet to the destination LEO domain. According to the inter-domain route, the agent then computes the intra-domain route to deliver the packet to the gateway or the destination within the domain. In normal scenarios, the inter-domain route is typically computed once by the first agent encountered when the packet enters the LEO constellation, and reused across domains unless significant topology changes are detected, while the intra-domain route is calculated by agents deployed at gateways in a distributed manner each time the packet enters a new domain. In abnormal scenarios, inter-domain and intra-domain route calculations may fail due to node failures. In such cases, upon detecting routing failure or topology inconsistency, the agent attempts to synchronize the inter-domain or intra-domain graph with neighboring nodes and recomputes the route. If the number of attempts exceeds the limit, the packet is finally dropped. Overall, routing proceeds iteratively across domains: once a packet reaches a domain boundary, the next domain is determined by the inter-domain route, and intra-domain routing is recomputed locally, until the destination domain is reached.
2) *Forwarding plane:* According to the route provided by the agent, under normal conditions, the router first checks whether the current node is the destination node. If so, the packet is directly delivered. Then, the router verifies whether the total hop count of the packet exceeds the limit; if it does, the packet is dropped. Finally, the router forwards the packet to the next-hop node. Under abnormal conditions (sudden node failures), the next hop may be inaccessible. In this case, the router buffers the packet and broadcasts a command to agents to restart the inter-domain or intra-domain routing computation and return the result. Since the route and control messages are much smaller than data packets and incur limited communication overhead, they are assumed to have a higher delivery reliability compared with data traffic.

III. Proposed Methdodology

In this section, we first introduce the embedding of the input routing requests. We then describe the TRACE architecture,

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

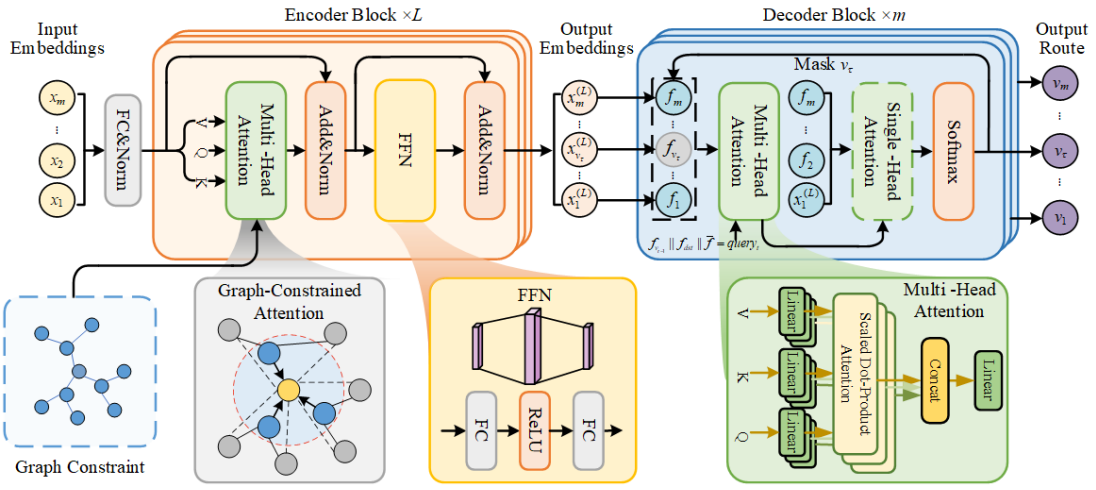


Fig. 4. The TRACE architecture. Node embeddings and graph relations are used respectively as the encoder input and as hard constraints in the attention computation, where all multi-head attentions in the proposed TRACE are computed under these graph constraints. The decoder's query vector is constructed by concatenating the features of the current node, the destination node, and the global state.

including the detailed design of the encoder and decoder. Finally, we propose the NFD framework to train a more efficient model.

A. Graph-Constrained Transformer Encoder

The inter-domain routing and intra-domain routing models share the same neural network architecture as shown in Figure 4. Taking the routing problem in a single domain with nodes set $\mathcal{V} = \{1, \ldots, m\}$ as an example, its topology is $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. The inputs are $\mathcal{G}$ and the corresponding communication delay $[T_{i,j}]_{m\times m} \in \mathbb{R}^{m\times m}$, source node src, and destination node dst. With these inputs, the i-th node input embedding x_i can be denoted by (15).

$$x_i = [\mathbb{I}(i = src),\ f(T_{i,1}),\ \ldots,\ f(T_{i,m}),\ \mathbb{I}(i = dst)], \quad (15)$$

where $\mathbb{I}(condition)$ is the indicator function, $\mathbb{I}(condition) = 1$ if $condition$ is true, otherwise, $\mathbb{I}(condition) = 0$. $f(x) = \min\{x,\ T_{max}\}$ is a truncation function, where T_{max} denotes the maximum response delay.

The propagation delay in LEO constellations exhibits a high dynamic range due to varying inter-satellite distances and dynamic link conditions. Directly using raw delay values may lead to unbalanced feature distributions and hinder effective model training. Therefore, the truncation function $f(x)$ is introduced to compress large delay values while preserving relative differences among smaller values, which are more critical for routing decisions. This type of feature scaling is widely adopted in graph representation learning and neural network models to improve numerical stability and training efficiency [34], [35].

The overall input features $x = [x_1, x_2, \ldots, x_m] \in \mathbb{R}^{m\times(m+2)}$ are embedded in d_x dimension features through a fully-connected layer (FC) before they are fed into the encoder blocks as denoted in (16).

$$x_i^{(0)} = LN(x_i \mathbf{W}_0 + \mathbf{b}), \quad (16)$$

where $LN(\cdot)$ represents layer normalization, $\mathbf{W}_0 \in \mathbb{R}^{(m+2)\times d_x}$ and $\mathbf{b} \in \mathbb{R}^{d_x}$ are learnable parameters. In the following, we denote the output of the l-th encoder block as $x^{(l)} = [x_1^{(l)}, \ldots, x_m^{(l)}]$, $x_i^{(l)} \in \mathbb{R}^{d_x}$. Then the input of the encoder is $x^{(0)}$, while the output is $x^{(L)}$.

Figure 4 shows the details of multi-head attention (MHA) block, which takes Q, K and V as input. If we define the feature dimension d_v of each head, then the number of heads $E = d_x/d_v$. For a single head, the Q, K, V can be denoted by (17):

$$Q = x^{(l-1)}\mathbf{W}_q,\ K = x^{(l-1)}\mathbf{W}_k,\ V = x^{(l-1)}\mathbf{W}_v, \quad (17)$$

where $\mathbf{W}_q$, $\mathbf{W}_k$ and $\mathbf{W}_v \in \mathbb{R}^{d_x\times d_v}$ are all learnable parameters.

Figure 4 illustrates the differences between our encoder and the Transformer. Unlike the positional encoding in Transformer, which emphasizes temporal correlations, graph optimization problems focus more on the relationships between nodes, represented by the constraint matrix $C = [c_{ij}]_{m\times m}$, where $c_{ij} = -\infty$ if $T_{i,j} = +\infty$, otherwise, $c_{ij} = 0$. It is worth noting that TRACE is not a generic graph-based Transformer model. In existing graph Transformer architectures (e.g., GTN [36]), attention mechanisms are typically used to learn latent relationships or augment node representations. In contrast, TRACE treats attention as a constrained decision operator, where the adjacency constraint matrix explicitly restricts attention computation to feasible routing neighbors. This design ensures that all attention operations are inherently aligned with routing feasibility, rather than relying on the model to implicitly learn graph connectivity.

By imposing graph constraints, the attention coefficient matrix $U \in \mathbb{R}^{m\times m}$ can be calculated with $softmax$ activation function by (18).

$$U = softmax[C + (QK^T)/\sqrt{d_v}]. \quad (18)$$

With attention coefficient matrices U_e and value matrices V_e from all heads $\{1, \ldots, E\}$, the output of the MHA block can be calculated by (19):

$$output = (||_{e=1}^{E} U_e V_e) \cdot \mathbf{W}_1, \quad (19)$$

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

where $\mathbf{W}_1 \in \mathbb{R}^{d_x \times d_x}$ is a learnable matrix, $||$ denotes the concatenation operation.

Through residual connection and layer normalization, we obtain $\hat{x}^{(l)} = LN(output + x^{(l-1)})$. The computed result is then fed into the feed-forward network (FFN), which consists of a $ReLU$ activation and two FC layers, i.e., learnable matrices $\mathbf{W}_2 \in \mathbb{R}^{d_x \times 2d_x}$ and $\mathbf{W}_3 \in \mathbb{R}^{2d_x \times d_x}$. Finally, a residual connection and layer normalization are applied to obtain the encoder output $x^{(l)}$. The detailed calculation is shown in (20).

$$x^{(l)} = LN[ReLU(\hat{x}^{(l)}\mathbf{W}_2) \cdot \mathbf{W}_3 + \hat{x}^{(l)}]. \tag{20}$$

B. Cascaded Mult-Head Attention Decoder

As shown in Figure 4, we employ a cascaded attention decoder similar to the structure proposed in [37], which enables Transformer-based models to output sequences with non-fixed length. Unlike standard Transformer decoders designed for sequence generation, the proposed cascaded attention decoder is specifically designed for step-wise routing decisions, leveraging task-conditioned query embeddings and enforcing strict feasibility constraints. Specifically, the MHA module first performs a coarse exploration over candidate next-hop nodes from multiple representation subspaces, while the subsequent single-head attention module refines this distribution to produce a more precise decision. This two-stage design enables more effective selection of next-hop nodes in highly dynamic graph environments, where both exploration and fine-grained discrimination are required.

Suppose that the output of the decoder is a route path $\mathbf{p} = [v_1, \ldots, v_\tau, v_m]$. The decoder block at time step $\tau, 2 \leq \tau \leq m$ takes $x^{(L)}$, $v_{\tau-1}$, and dst as inputs, and outputs the hop decision v_τ. To ensure consistency, we define the output of the decoder at the first time step $v_1 = src$.

To distinguish it from the encoder, we use $f = [f_1, \ldots, f_m]$ instead of $x^{(L)}$ to denote the node feature embedding in the decoder. Then the input q, K^d, V^d of the MHA in the τ-th decoder block ($2 \leq \tau \leq m$) can be calculated by (21):

$$q_\tau = query_\tau \cdot \mathbf{W}_q^d,\ K^d = f\mathbf{W}_k^d,\ V^d = f\mathbf{W}_v^d, \tag{21}$$

where $\mathbf{W}_q^d$, $\mathbf{W}_k^d$ and $\mathbf{W}_v^d \in \mathbb{R}^{d_x \times d_v}$ are all learnable parameters. It is worth mentioning that K^d and V^d are computed only once in the first decoder block and kept fixed afterward. $query_\tau = (f_{v_{\tau-1}}||f_{dst}||\bar{f})$ is used to describe the current routing problem, where $f_{v_{\tau-1}}$ is the current node feature, f_{dst} is the dst node feature, and $\bar{f} = \frac{1}{m}\sum_{i=1}^{m} f_i$ represents the graph topology.

Then, the attention vector $u^\tau \in \mathbb{R}^m$ can be calculated by (22):

$$u^\tau = softmax[Mask_\tau + (q_\tau K^T)/\sqrt{d_v}], \tag{22}$$

where $Mask_t = [b_1(\tau), \ldots, b_i(\tau), \ldots, b_m(\tau)]$, and $b_i(\tau)$ is given as (23). $Mask_\tau$ not only ensures that the path is loop-free as (14d), but also guarantees that the selected next-hop node conforms to the graph constraints.

$$b_i(\tau) = \begin{cases} -\infty, & i \notin \mathcal{N}(v_{\tau-1}) \text{ or } \exists \tau' < \tau,\ v_{\tau'} = i. \\ 0, & \text{otherwise.} \end{cases} \tag{23}$$

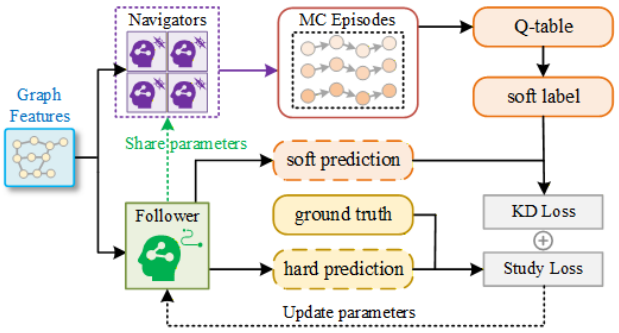


Fig. 5. The navigator-follower distillation framework.

Similarly, we give the MHA output in the τ-th decoder block as (24).

$$output^\tau = (||_{e=1}^{E} u_e^\tau V_e^d) \cdot \mathbf{W}_1^d, \tag{24}$$

where $\mathbf{W}_1^d \in \mathbb{R}^{d_x \times d_x}$ is a learnable matrix, u_e^τ and V_e^d are the attention coefficient vector and the value matrix from head e respectively. Then $output^\tau$ is fed into the single-head attention block as a more detailed query vector. Since single-head attention is a special case of MHA with $E = 1$ as (21), we do not elaborate on its computation process. Instead, its operation is defined through the function $sha(\cdot)$. Finally we have the output of the τ-th decoder block as (25).

$$\pi_\phi^\tau = softmax[sha(output^\tau)], \tag{25}$$

where $v_\tau \sim \pi_\phi^\tau(v_\tau)$, and $\mathbf{p} \sim \pi_\phi(\mathbf{p}|s), s = \langle src, dst, \mathcal{G} \rangle$. v_τ is selected by random sampling during training, whereas a greedy strategy is used during inference. The decoding process terminates when $v_\tau = dst$.

This cascaded attention mechanism differs from conventional decoding strategies by explicitly separating candidate exploration and decision refinement, rather than directly generating outputs in a single attention step.

C. Navigator-Follower Distillation Mechanism

To discover high-quality routing paths, DRL-based approaches typically rely on repeated environment interactions and policy updates. Prior studies have pointed out that, in wireless network control problems, DRL often faces slow convergence and performance instability [38], [39]. However, routing in LEO constellations is not a purely trial-and-error problem. For a given topology, high-quality routing paths can be efficiently obtained using classical algorithms such as Dijkstra, which provide strong supervision signals.

Motivated by this observation, we propose the Navigator–Follower Distillation (NFD) framework, which integrates ground-truth supervision with Monte Carlo-based policy evaluation to achieve efficient and stable routing policy learning.

We train both the inter-domain and intra-domain routing models under the NFD framework to enhance their generalization capability. In the following, we will use the intra-domain model as an example to illustrate the training and inference process. As shown in Figure 5, the NFD framework consists

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

of two major parts, one part is to calculate the study loss, the other part is to calculate the knowledge distillation (KD) loss.

Intuitively, the study loss and the KD loss play complementary roles. The study loss provides supervision from ground-truth paths and ensures that the model learns valid routing trajectories. In contrast, the KD loss leverages Monte Carlo episodes to evaluate the relative quality of candidate next hops, enabling the model to capture fine-grained differences between hops that cannot be distinguished by ground truth alone.

The study loss is consistent with the standard teacher-student distillation framework, which is a supervised loss function. Given the ground-truth path $\mathbf{p}^g = [v_1^g, \ldots, v_\tau^g, \ldots, v_m^g]$ for a routing request req in graph $\mathcal{G}$, the study loss $\mathcal{L}_{SL}$ can be computed through the discounted likelihood function, as derived in (26)

$$\mathcal{L}_{SL}(\mathbf{p}^g; \phi) = \sum_{\tau=2}^{m} \gamma^{\tau-2} \ln \pi_\phi^\tau(v_\tau^g), \tag{26}$$

where $0 < \gamma < 1$ is a discount factor, which enables the model to capture the dependency between hop decisions.

However, the study loss does not enable the agent to perceive the correctness of each hop. Even if two hops on the same path are both consistent with the ground truth, such a supervised learning–based loss function cannot capture their respective contributions to the total delay. Therefore, we adopt a self-distillation approach to compute the KD loss, where one follower and N navigators work in parallel.

Unlike standard DRL paradigms, NFD decouples trajectory generation from model optimization, rather than learning policies through coupled actor–critic updates or policy-gradient optimization. The navigators perform Monte Carlo sampling to estimate action-quality information in a lightweight manner, which avoids the need for value-function approximation and repeated bootstrapped updates commonly required in DRL methods.

Algorithm 1 Q-Table Calulating With Monte Carlo Episodes

Input: routing case s; parameters $\tilde{\phi}$; discount factor γ; navigators number N.
Initialization: $\mathcal{Q} \leftarrow \mathbf{0}$, $\mathbf{C} \leftarrow \mathbf{0}$.
  for $i = 1, \ldots, N$ **do**
    $sum \leftarrow 0$
    the i-th navigator sample $MC_i \sim \pi_{\tilde{\phi}}(MC_i|s)$
    for $j = |MC_i|, \ldots, 2$ **do**
      $u \leftarrow v_{i,j-1}^{MC}$, $v \leftarrow v_{i,j}^{MC}$
      $sum \leftarrow \gamma \cdot sum - D(u, v)$
      $\mathbf{C}_{u,v} \leftarrow \mathbf{C}_{u,v} + 1$
      $\mathcal{Q}_{u,v} \leftarrow \mathcal{Q}_{u,v} + \frac{sum - \mathcal{Q}_{u,v}}{\mathbf{C}_{u,v}}$
    end for
  end for
Output: $\mathcal{Q}$

During the NFD training procedure, the follower shares the parameters $\tilde{\phi}$ from the previous epoch with navigators. The navigators generates N Monte Carlo episodes by stochastic sampling and uses them to estimate the Q-table $\mathcal{Q}(s; \tilde{\phi}) \in \mathbb{R}^{V \times V}$, where $s = \langle req, \mathcal{G} \rangle$. We define the i-th Monte Carlo episode as $MC_i = [v_{i,1}^{MC}, v_{i,2}^{MC}, \ldots]$, the Q-table can be calculated by Algorithm 1.

It is reasonable to assume that a next hop with a larger Q-value should be more likely to be selected, i.e., the selection probability $\pi_{\mathcal{Q}}^{\tau+1}(v_{\tau+1}) = \frac{exp[\mathcal{Q}_{v_\tau, v_{\tau+1}}(s;\tilde{\phi})]}{\sum_{j \in \mathcal{N}(v_\tau)} exp[\mathcal{Q}_{v_\tau, j}(s;\tilde{\phi})]}$. If the Q-routing path is $\mathbf{p}^q = [v_1^q, \ldots, v_\tau^q, \ldots, v_m^q]$, we can compute the KD loss by (27):

$$\mathcal{L}_{KD}(\mathbf{p}^q; \phi) = \sum_{\tau=2}^{m} \gamma^{\tau-2} KL(\pi_{\mathcal{Q}}^\tau || \pi_\phi^\tau), \tag{27}$$

where $KL(\pi_{\mathcal{Q}}^\tau || \pi_\phi^\tau) = \sum_{j \in \mathcal{N}(v_{\tau-1}^q)} \pi_{\mathcal{Q}}^\tau(j) \cdot \ln \frac{\pi_{\mathcal{Q}}^\tau(j)}{\pi_\phi^\tau(j)}$ denotes the Kullback-Leibler (KL) divergence.

We finally update the parameter ϕ by (28), where lr is the learning rate, and $\alpha_1,\ \alpha_2 > 0$, $\alpha_1 + \alpha_2 = 1$,

$$\begin{aligned}\phi = \phi &+ lr \cdot \alpha_1 \mathbb{E}_{\mathbf{p} \sim \pi_\phi(\mathbf{p}|s)}[\nabla_\phi \mathcal{L}_{SL}(\mathbf{p}; \phi)] \\ &+ lr \cdot \alpha_2 \mathbb{E}_{\mathbf{p} \sim \pi_{\mathcal{Q}}(\mathbf{p}|s)}[\nabla_\phi \mathcal{L}_{KD}(\mathbf{p}; \phi)].\end{aligned} \tag{28}$$

Algorithm 2 NFD Training Procedure

Input: number of epochs $Epochs$; steps per epoch $Steps$; batch size $\mathcal{B}$; dataset **data**; navigators number N; learning rate lr; parameters ϕ; discount factor γ; loss function coefficient α_1, α_2.
Initialization: Orthogonal initialization θ, ϕ; set $\tilde{\phi}$ as uniformly sampling; $\mathcal{Q}_i = \mathbf{0}, \forall s_i \in \mathbf{data}$.
  Navigators:
    wait for new $\tilde{\phi}$
    for $s_i \in \mathbf{data}$ **do**
      perform Algorithm 1 and update $\mathcal{Q}_i$
    end for
  Follower:
    for $epoch = 1, \ldots, Epochs$ **do**
      for $step = 1, \ldots, Steps$ **do**
        select $s_i \in \mathbf{data}, \forall 1 \le i \le \mathcal{B}$
        get $\mathbf{p}_i^q$ precomputed via Dijkstra's algorithm
        compute $\mathbf{p}_i^q$ with $\mathcal{Q}_i$ via Q-routing
        compute $\mathcal{L}_{SL}(\mathbf{p}_i^q; \phi)$ via (26)
        compute $\mathcal{L}_{KD}(\mathbf{p}_i^q; \phi)$ via (27)
        $\phi \leftarrow \phi + lr \cdot \frac{1}{\mathcal{B}} \nabla_\phi [\alpha_1 \mathcal{L}_{SL} + \alpha_2 \mathcal{L}_{KD}]$
      end for
      $\tilde{\phi} \leftarrow \phi$, send $\tilde{\phi}$ to navigators
    end for
  Output: ϕ

The detailed NFD training procedure for multi-domain routing is shown as Algorithm 2, where **data** denotes the training dataset constructed as described in Section IV-A. In addition, the expectations mentioned in (28) are approximated via batch sampling.

To further analyze the computational characteristics of different training paradigms, we summarize their training complexity in Table I. In Table I, *Steps* denotes the number of training iterations per epoch, and $\mathcal{B}$ is the batch size. $\bar{L}$ represents the average routing path length, and N is the number of navigators used in the NFD framework. C_f and C_b denote the

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TRAINING COMPLEXITY COMPARISON

Method	Signal	Complexity	Loop
SUP	GT	$O(Steps\mathcal{B}(C_f + C_b))$	×
NFD	GT + Q	$O(Steps\mathcal{B}(C_f + C_b)) + O(Steps\mathcal{B}\bar{L}N)$	×
PPO	Reward	$O(Steps\mathcal{B}C_f) + O(E_p \cdot Steps\mathcal{B}(C_f + C_b))$	√

forward and backward computation cost of the neural model, respectively. E_p denotes the number of optimization epochs in PPO for each batch of collected samples. The ground-truth (GT) paths are precomputed using Dijkstra's algorithm and are not included in the training complexity.

For supervised learning (SUP), the training cost is dominated by the forward and backward propagation of the neural model, resulting in a complexity of $O(Steps\mathcal{B}(C_f + C_b))$.

For the proposed NFD framework, the overall training cost consists of the follower-side optimization and the navigator-side evaluation. The follower optimization has the same order as SUP, i.e., $O(Steps\mathcal{B}(C_f + C_b))$, while the navigator component introduces an additional cost of $O(Steps\mathcal{B}\bar{L}N)$ for Monte Carlo trajectory sampling and Q-table estimation. Therefore, the total complexity of NFD can be written as $O(Steps\mathcal{B}(C_f + C_b + \bar{L}N))$.

For PPO, the training complexity includes both trajectory sampling and iterative policy optimization. The sampling stage incurs a cost of $O(Steps\mathcal{B}C_f)$, while the collected samples are further processed through E_p optimization epochs, leading to an additional cost of $O(E_p \cdot Steps\mathcal{B}(C_f + C_b))$. Thus, the overall complexity of PPO can be expressed as $O(Steps\mathcal{B}(C_f + E_p(C_f + C_b)))$, which is dominated by the repeated optimization term when $E_p > 1$.

Compared with PPO, NFD avoids repeated inner-loop optimization over the same batch of data and instead introduces a lightweight navigator-side evaluation mechanism. Furthermore, the follower optimization and navigator-side evaluation can be executed in parallel in practice, which reduces the overall wall-clock overhead. As a result, NFD provides a more efficient and stable training process, particularly under highly dynamic routing conditions, which is further reflected in the faster convergence and improved performance observed in the experimental results.

IV. EXPERIMENTS AND NUMERICAL RESULTS

A. Experimental Setup

We train our routing models under three different single-domain network scales: D100 (100-node domain), D50 (50-node domain), and D30 (30-node domain). We further construct a 20-domain LEO constellation topology, denoted as C20. We generate the datasets accordingly: the D100 and D50 datasets each contain 512,000 training samples and 10,000 validation samples; the D30 dataset contains 256,000 training samples and 5,000 validation samples; and the C20 dataset includes 128,000 training samples and 2,500 validation samples. We adopt the LEO constellation parameters that come from or are derived from [19], [40], [41], [42], which are detailed in Table II.

TABLE II
PARAMETERS

Category	Parameter	Value
LEO Constellation	Transmission power P	0.7 W
	Channel gain related constant C	0.8
	ISL bandwidth B	20 GHz
	ISL distance mean d	1,000 km
	ISL distance variance σ_d^2	200
	Noise density ρ	-200 dBW/Hz
Packet Routing	Transimitted user packet κ	4 MB
	Maximum queue capability $queue_{max}$	32 MB
	Hop limit H	15
	Try limit TRY	7
Model Training	Number of epochs $Epochs$	100
	Batch size $\mathcal{B}$	512
	Navigators number N	8
	Learning rate lr	1×10^{-4}
	Discount factor γ	0.95
	Loss factor α_1, α_2	0.3, 0.7

100-node routing models are trained and validated on a server with the Ubuntu 20.04 operating system, equipped with 256GB of memory, an Intel(R) Xeon(R) Gold 5218 CPU, and two NVIDIA RTX4090 GPUs; while 50-node and 30-node routing models are trained and validated on a PC with the Windows 11 operating system, equipped with 64GB of memory, an AMD Ryzen 9 9950×3D CPU, and one NVIDIA RTX 5080 GPU. Table II shows the detailed hyperparameters.

Figure 6 shows the examples of D30, D50, D100, and C20, where yellow nodes are the source nodes, red nodes are the destination nodes, gray nodes are the failed nodes, and the green arrows depict the valid paths. The average degree of LEO satellites in D30, D50 and D100 are set to 4 corresponding to the mesh topology, while it is set to 8 in C20. Moreover, we set the maximum node failure percentage $p_{fail}^{max} = 0.382$ (where the actual failure percentage follows a uniform distribution, i.e., $p_{fail} \sim U(0, p_{fail}^{max})$), and the node failure mode to *Targeted Attack* during training, while the other possible values of p_{fail}^{max} and the *Random Failure* mode are evaluated during inference. Since it is nearly impossible for all gateway nodes to fail simultaneously, C20 typically involves only edge failures. To enhance the inter-domain routing model's performance, edge failures in C20 are generated based on ground-truth path lengths from 1 to 20, where no edge fails in ground-truth paths.

B. Baselines

As shown in Table III, we evaluate the proposed NFD and TRACE against several commonly used baselines. The deterministic group includes Dijkstra and Δ-stepping [43]. Both offer exact shortest-path results, but their scalability is limited, and only Δ-stepping supports partial parallel execution. For heuristic search, we consider Limited Discrepancy Search (LDS) [44], which explores a restricted discrepancy space and serves as a lightweight inference method. The learning-based group includes Graph Transformer Network (GTN) [36], GAT [45], and TRACE, all of which use topology-aware representations and provide strong baselines for dynamic routing. For training schemes, we further compare three setups:

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

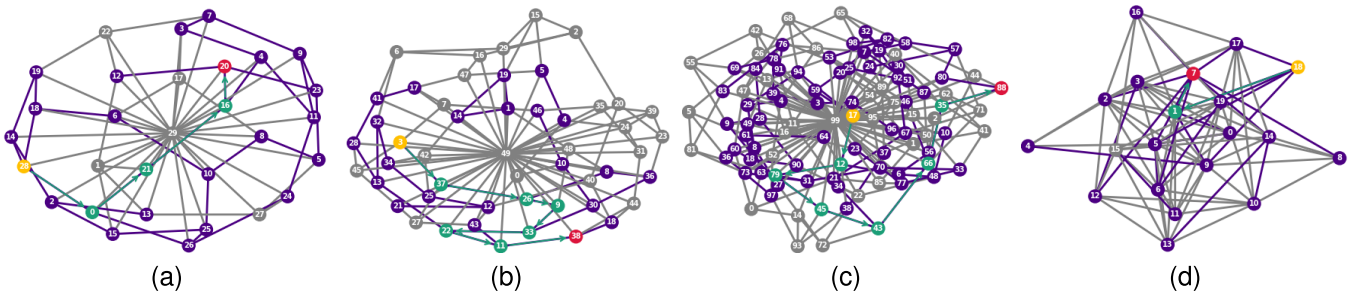


Fig. 6. Examples of samples in datasets. (a) An example of D30. (b) An examples of D50. (c) An example of D100. (d) An example of C20.

TABLE III
BASELINES

Category	Method	Type	Parallel
Routing Inference	Dijkstra	Deterministic	×
	Δ-stepping	Deterministic	√
	LDS	Heuristic	×
	GTN	Nerual Network	√
	GAT	Nerual Network	√
	TRACE	Nerual Network	√
Training	NFD	Semi-Supervised	–
	PPO	Reinforced	–
	SUP	Supervised	–

TABLE IV
TRAINING RESULTS

Model	Random Failure		Targeted Attack	
	Delay (ms)	Loss Rate (%)	Delay (ms)	Loss Rate (%)
PPO-6	280.88	9.01	1060.06	46.50
SUP-6	259.60	7.41	1016.12	43.79
SUP-4	287.82	8.72	1092.74	46.29
NFD-4	**240.29**	**6.83**	**1018.31**	**43.64**

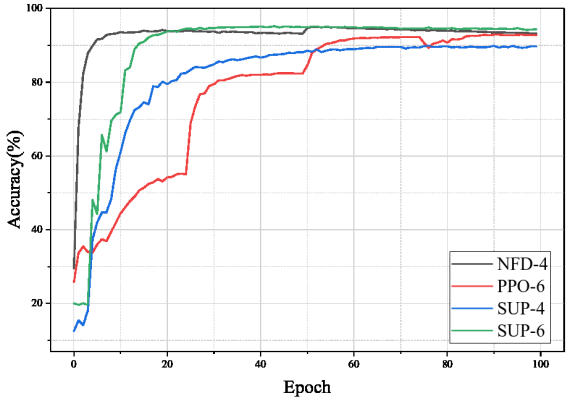


Fig. 7. Validation accuracy over epochs for TRACE through different training methods.

semi-supervised NFD, reinforcement-learning PPO [46], and supervised SUP. This setup provides a balanced view across the mainstream algorithms.

C. Metrics

In our evaluation, we assess the proposed methods from four perspectives. First, we measure the packet loss rate to capture the impact of routing decisions under node failures. Second, we evaluate end-to-end latency, which reflects the efficiency of the computed paths. Third, we report algorithm throughput, defined as the number of routing requests the model can process per second. Finally, we quantify reliability using the ratio between the algorithm's delivery success rate and the ground-truth delivery rate. These metrics jointly reflect optimality, efficiency, and resilience under large-scale and highly dynamic LEO constellations.

D. Training Methods Comparison

In this experiment, we compare the trained TRACE models with different training methods. To enhance the validity of the conclusions, we compare four configurations: NFD-4 (trained with NFD and a 4-layer encoder), PPO-6 (trained with PPO and a 6-layer encoder), SUP-4 (trained with SUP and a 4-layer encoder), and SUP-6 (trained with SUP and a 6-layer encoder).

The overall results presented in Table IV show the average delay (in ms) and loss rate (in %) for each model under these two scenarios, validated on the D100 dataset and $p_{fail}^{max} = 0.5$. We observe that NFD-4 outperforms the other models in both random failure and targeted attack scenarios, with the lowest delay of 240.29 ms and the lowest loss rate of 6.83% under random failures, and a delay of 1018.31 ms and a loss rate of 43.64% under targeted attacks. In comparison, SUP-6 and SUP-4 have slightly higher delays and loss rates, while PPO-6 shows the highest values across both metrics.

Comparing NFD-4 with SUP-4, it is evident that NFD achieves a better routing strategy with the same parameter scale. Comparing NFD-4 with SUP-6, NFD enables a model with fewer parameters to achieve performance that is comparable to, or even exceeds, that of a larger model. A comprehensive comparison of NFD-4, SUP-6, and PPO-6 shows that as a semi-supervised method, NFD combines the reward feedback from reinforcement learning and the ground truth guidance from supervised learning, making it more effective in training optimal parameters.

Figure 7 shows the training curves for the four training methods, where the y-axis represents the accuracy, defined as the delay of the true path divided by the delay of the model's output path. Notably, due to the differences in the loss

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

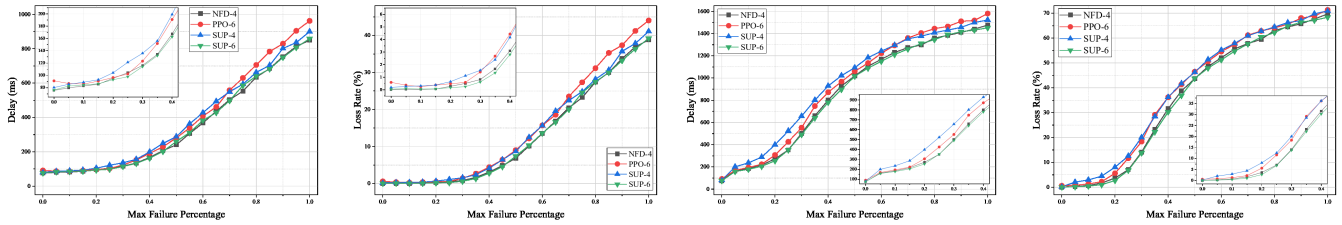


Fig. 8. Performance evaluation of the trained models under random failures and targeted attacks. (a) and (b) are the delay and loss rate comparisons of different training methods under random failures, while (c) and (d) are the comparisons under targeted attacks.

functions used by the four methods, we do not use the loss value as the y-axis. From the figure, it can be seen that all four methods converge. In terms of convergence results, NFD-4 and SUP-6 achieve the highest accuracy, followed by PPO-6 and SUP-4, which is consistent with the results in Table IV. In terms of convergence speed, NFD-4 achieves the fastest convergence, thanks to its self-distillation framework. SUP-6, with its larger parameter scale, converges next, as it can learn from the ground truth more quickly. SUP-4, despite having ground truth guidance, cannot fully fit the optimal parameter distribution due to its limited parameters. PPO-6, lacking ground truth guidance, converges the slowest, with its curve showing a stepwise pattern due to the need for path exploration.

Figure 8 presents inference results under random failures and targeted attacks, with respect to $0 \leq p_{fail}^{max} \leq 1$. Figure 8 (a) and Figure 8 (b) show the performance under random failures. As p_{fail}^{max} increases, the delay and loss rate for all methods rise. NFD-4 and SUP-6 maintain the lowest delay and loss rate across the entire range, demonstrating superior resilience to failure. In contrast, PPO-6 and SUP-4 exhibit significantly higher delay and loss rate, especially as failure percentages approach 100%. Figure 8 (c) and Figure 8 (d) show results under targeted attacks. The performance degradation is more pronounced than under random failures, especially in delay and loss rate. NFD-4 and SUP-6 again outperform all other methods, maintaining the lowest delay and loss rate. PPO-6 and SUP-4 experience rapid increases in both delay and loss rate under higher failure percentages.

Overall, NFD-4 achieves the best performance under both random failures and targeted attacks with the fewest parameters, demonstrating the effectiveness of NFD in training routing models for dynamic LEO networks.

E. Inference Performance Comparison

This experiment focuses on evaluating the optimality of each method, we compare TRACE with several mainstream routing methods including GAT, GTN, LDS and Dijkstra in dataset D50, where the Dijkstra is treated as the ground truth. To ensure a fair comparison, all neural network–based methods are configured with a 6-layer encoder and trained by SUP. In addition, since Δ-stepping is a parallelized edition of Dijkstra, it is not included in the comparison.

To illustrate the routing behavior of different algorithms, Fig. 10 presents a representative case in which all methods

TABLE V
INFERENCE PERFORMANCE

Method	Random Failure		Targeted Attack	
	Delay (ms)	Loss Rate (%)	Delay (ms)	Loss Rate (%)
GAT	246.80	8.13	861.12	37.69
GTN	359.49	11.49	1067.80	44.22
LDS	667.94	29.26	1082.63	50.3
TRACE	**238.70**	**7.9**	**886.28**	**38.79**
Dijkstra	228.92	7.46	853.50	37.33

successfully deliver the packet. TRACE infers the exact same route as Dijkstra, demonstrating its optimality. In contrast, GAT selects a longer route, resulting in a delay of 268 ms. GTN and LDS infer similar routes, differing only at the second-to-last hop: GTN selects node 45, whereas LDS selects node 17. This difference leads to GTN achieving a lower delay (245 ms) compared with LDS (279 ms).

Table V summarizes the inference performance under both random failures and targeted attacks with $p_{fail}^{max} = 0.5$. Under random failures, TRACE achieves the best overall performance among the learning-based methods, reducing delay relative to GAT and GTN while maintaining the lowest loss rate except for Dijkstra. LDS performs significantly worse, with both delay and loss rate growing sharply. In contrast, under targeted attacks, GAT shows the strongest robustness among neural models, achieving the lowest delay (861.12 ms) and loss rate (37.69%). TRACE ranks second, only slightly worse than GAT and remaining close to the Dijkstra ground-truth, with a 9.78 ms delay gap and a 1.46% loss-rate gap. GTN and LDS degrade substantially under targeted attacks, indicating weaker resilience. Overall, TRACE maintains strong performance across both scenarios, while GAT exhibits a distinct advantage under targeted attacks. However, as p_{fail}^{max} increases, GAT exhibits pronounced instability, as shown in Figure 9. The detailed performance of each method under different levels of node failures will be analyzed in the following.

Under random failures, the delay curves in Figure 9 (a) show that TRACE and Dijkstra remain nearly identical across all failure levels, while GTN exhibits moderately higher delay, whereas LDS degrades rapidly as failures increase. GAT maintains low delay only when $p_{fail}^{\max}$ is small, but becomes unstable and deteriorates sharply once the failure ratio enters the mid-range. The loss rate results in Figure 9 (b) reveal a different tendency. The loss rate of GAT does not grow proportionally with $p_{fail}^{\max}$, indicating that large failure ratios disrupt

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

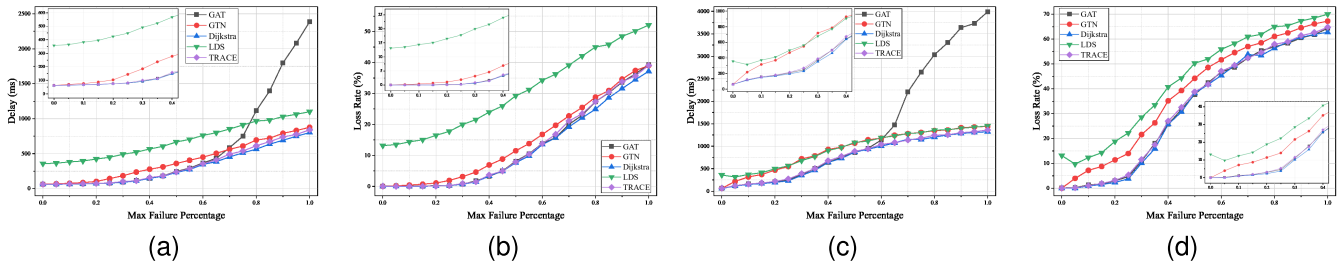


Fig. 9. Inference performance comparisons of different routing methods under random failures and targeted attacks. (a) and (b) are the delay and loss rate comparisons of different routing methods under random failures, while (c) and (d) are the comparisons under targeted attacks.

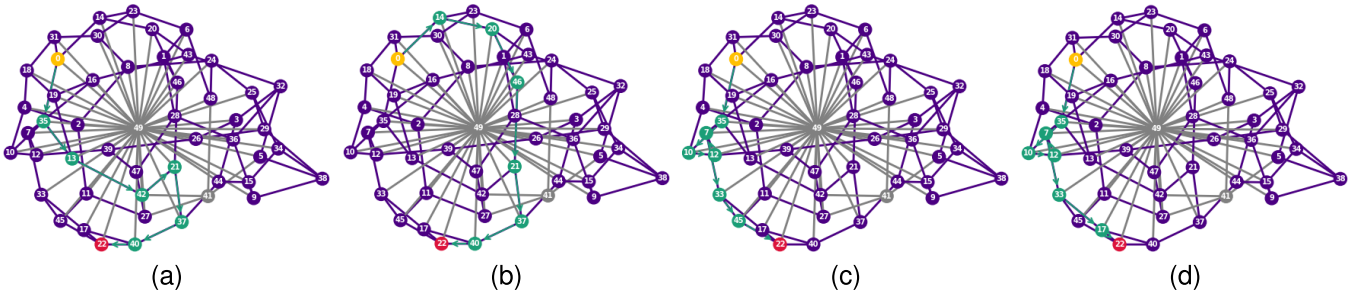


Fig. 10. A representative routing case where all methods achieve successful delivery. (a) TRACE and Dijkstra infer the same route $[0, 35, 13, 42, 21, 37, 40, 22]$ with a delay of 218 ms; (b) GAT infers the route $[0, 14, 20, 46, 21, 37, 40, 22]$ with a delay of 268 ms; (c) GTN infers the route $[0, 35, 10, 12, 7, 33, 45, 22]$ with a delay of 245 ms; (d) LDS infers the route $[0, 35, 10, 12, 7, 33, 17, 22]$ with a delay of 279 ms.

its optimality: even when packets are successfully delivered, GAT increasingly selects unnecessarily longer routes. TRACE and Dijkstra consistently achieve the lowest loss rates, while GTN grows steadily and LDS shows the most severe degradation.

Under targeted attacks, the delay results in Figure 9 (c) show that GAT achieves the lowest delay when $p_{fail}^{\max}$ is small, but its delay rises sharply once the attack scale increases. Importantly, this degradation is not caused by routing failures on critical-node removal; rather, as confirmed by the loss-rate curves in Figure 9 (d), GAT maintains a relatively normal delivery ratio even under high failure levels. The divergence between Figure 9 (c) and (d) indicates that, under large targeted-attack magnitudes, GAT tends to select significantly longer surviving paths despite being able to deliver packets, revealing a loss of routing optimality rather than reliability.

This inconsistency between delay and loss-rate trends reflects GAT's sensitivity to structured disruptions: once key nodes fail, its attention mechanism becomes biased toward suboptimal long-range routes, increasing delay while keeping loss rate relatively stable. In contrast, TRACE and Dijkstra remain tightly aligned across all failure scales, demonstrating strong robustness and consistent path optimality. An additional observation is that LDS performs better under targeted attacks than under random failures. Since targeted attacks have removed key nodes, the remaining network exhibits a more coherent structural pattern, and hop decisions made by LDS are strongly influenced by static graph features—benefits from this increased structural regularity. Overall, targeted attacks expose GAT's instability in maintaining optimality, while TRACE preserves both optimality and reliability across varying failure scales.

TABLE VI
MULTI-DOMAIN PERFORMANCE

Method	D30 + C20		D50 + C20		D100 + C20	
	S (%)	R (%)	S (%)	R (%)	S (%)	R (%)
LDS	-96.59	96.42	-97.10	90.69	-98.11	80.90
GAT-SUP-6	13.55	99.67	-33.77	99.74	-20.27	99.34
GTN-SUP-6	51.11	97.52	-7.31	96.05	4.18	92.69
TRACE-SUP-6	48.56	99.95	-10.81	99.85	1.19	99.66
TRACE-NFD-4	**100.85**	**99.83**	**23.35**	**99.79**	**41.88**	**99.66**
Δ-stepping	0.00	100	0.00	100	0.00	100

F. Multi-Domain Performance Evaluation

This experiment evaluates the throughput and reliability of several mainstream routing algorithms in multi-domain scenarios. We compare TRACE-NFD-4 against GAT-SUP-6, GTN-SUP-6, LDS, and Δ-stepping in C20 combined with D30, D50, and D100 under targeted attacks with $p_{fail}^{\max} = 0.05$, where Δ-stepping serves as the baseline. Taking GAT-SUP-6 as an example, the first component (GAT) denotes the model, the second component (SUP) specifies the training method, and the final number (6) indicates the encoder depth.

Table VI reports the average throughput gain (S) and routing relative reliability (R) of each method in the C20–D30/D50/D100 multi-domain setting, where S denotes the percentage of throughput improvement over the Δ-stepping baseline and R denotes the reliability ratio of packet delivery rates relative to the baseline. Taking TRACE-NFD-4 ($S = 100.85\%$) as an example, if Δ-stepping processes 50000 samples per second, then the TRACE-NFD-4 can process $(1 + S) * 50000 = 100,425$ samples per second. The results show that the proposed TRACE variants, especially TRACE-NFD-4, consistently outperform other methods across all domain sizes in terms of both throughput and reliability.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

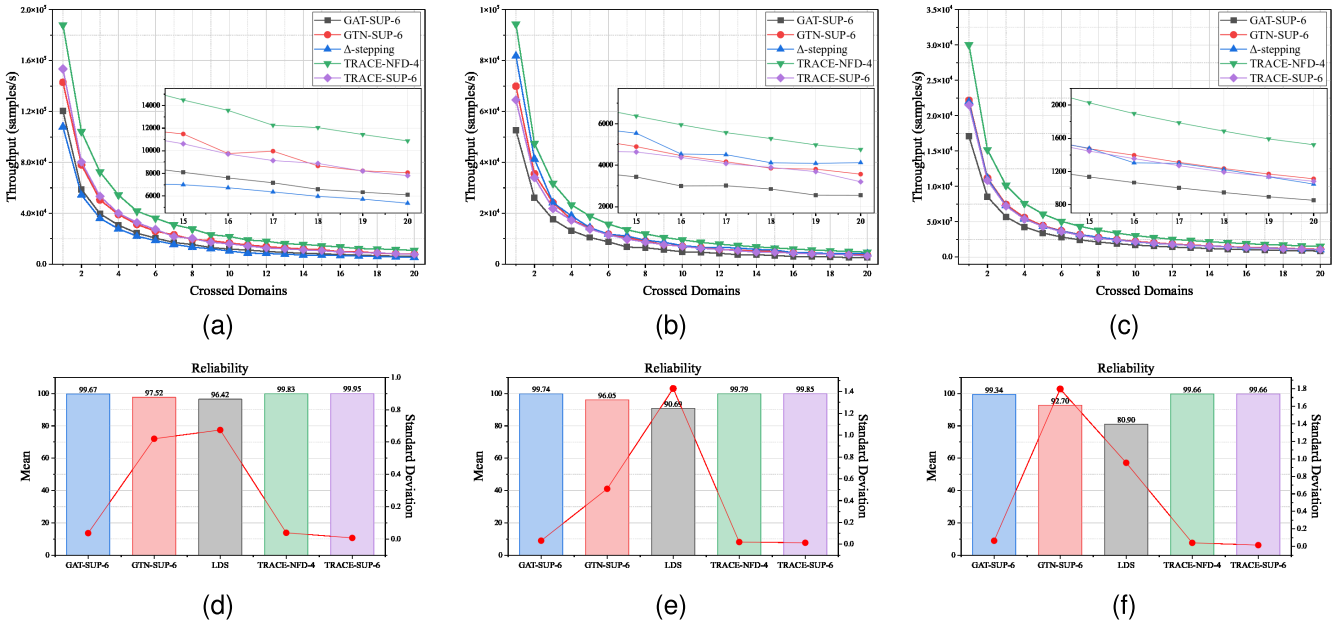


Fig. 11. Multi-domain routing performance evaluation of different routing methods under $p_{fail}^{\max} = 0.05$ targeted attacks. (a) and (d) are the throughput and reliability comparisons of different routing methods in D30+C20, (b) and (e) are the comparisons in D50+C20, while (c) and (f) are the comparisons in D100 +C20.

In D30, all neural models achieve positive throughput gains over Δ-stepping, while LDS remains far below the baseline. This shows that, in small domains, the structural complexity of routing tasks is limited and the models can efficiently exploit local patterns, allowing efficient parallel inference. TRACE-SUP-6 reaches a 48.56% throughput gain with near-perfect reliability (99.95%), and TRACE-NFD-4 further improves the gain to 100.85% with similarly high reliability (99.83%). GTN-SUP-6 also performs competitively with a 51.11% throughput improvement and 97.52% reliability.

As the domain size increases to D50, all neural models except TRACE-NFD-4 exhibit negative throughput gains, despite maintaining high reliability. This phenomenon reflects a scale-dependent effect: medium-sized domains introduce more irregular and less predictable routing structures, reducing the parallel inference effectiveness. In this case, the routing workload becomes less aligned with the models' strengths, whereas Δ-stepping operates closer to its most favorable condition, leading to a temporary throughput reversal. LDS remains both slower and less reliable, illustrating its poor adaptability to multi-domain routing.

When the domain size further increases to D100, throughput recovers for several neural models. GTN-SUP-6 and TRACE-SUP-6 return to positive throughput gains (4.18% and 1.19%, respectively), and TRACE-NFD-4 maintains the best (23.35% in D50 and 41.88% in D100) while sustaining reliability above 99.6%. This phenomenon suggests that larger domains provide richer redundancy and more consistent cross-domain routing patterns, enabling neural networks to better exploit large-scale structural cues.

Figure 11 (a)–(c) show the throughput trends of different routing methods under C20 combined with D30, D50, and D100. Consistent with the table results, TRACE-NFD-4 maintains the highest throughput across all three settings, while TRACE-SUP-6 and GTN-SUP-6 form a second tier and outperform Δ-stepping. GAT-SUP-6 yields the lowest platform throughput among neural models, as it tends to select longer paths under dynamic conditions. For clarity, LDS is omitted in Figure 11 (a)–(c) due to its extremely low throughput.

The influence of domain size is also observed from the throughput curves. In D30 +C20 (Figure 11 (a)), all neural models surpass Δ-stepping, indicating that their learned routing policy can effectively leverage the patterns in small-scale domains. In D50 +C20 (Figure 11 (b)), all neural models except TRACE-NFD-4 fall below the baseline. This behavior reflects that the routing patterns in D50 align less favorably with neural inference, whereas Δ-stepping performs closer to its most beneficial operating condition.

In contrast, under the larger D100 +C20 topology (Figure 11 (c)), throughput partially recovers for several neural models. This recovery suggests that large domains restore more stable long-range routing patterns and richer structural redundancy, which can again be effectively captured and exploited during distributed inference. As a result, TRACE-NFD-4 maintains the highest throughput across all hop distances in D100 +C20, indicating its scalability advantage in multi-domain routing environments.

Figure 11 (d)–(f) present the corresponding reliability results averaged over 20 cross-domain configurations. TRACE-NFD-4 achieves near-optimal mean reliability with the smallest variance in all three settings, showing strong robustness under heterogeneous domain structures. TRACE-SUP-6 and GAT-SUP-6 maintain high reliability with slightly higher variance, while GTN-SUP-6 shows moderate reliability and greater sensitivity to cross-domain hops. LDS exhibits

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

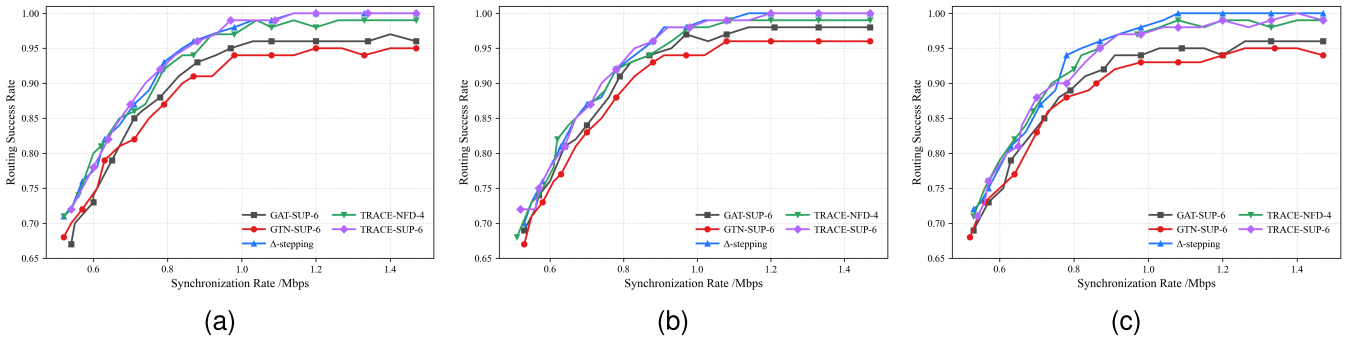


Fig. 12. Routing success rate versus synchronization rate under constrained inter-domain signaling. (a) D30 +C20, (b) D50 +C20, and (c) D100 +C20.

both lower reliability and the largest variance, confirming its instability in multi-domain distributed routing.

Overall, this experiment shows that TRACE-NFD-4 is the only method that achieves the highest throughput and near-optimal reliability, which is due to the proposed routing framework: intra-domain dynamics do not affect inter-domain routing decisions, and agents can resynchronize network topologies and recompute routes when failures occur, ensuring that routing errors remain controlled across domains.

TABLE VII
SYNCHRONIZATION THRESHOLD FOR STABLE ROUTING

Method	D30 + C20	D50 + C20	D100 + C20
	R_{sync}^{95} (Mbps)	R_{sync}^{95} (Mbps)	R_{sync}^{95} (Mbps)
GAT-SUP-6	0.97	0.93	1.03
GTN-SUP-6	1.40	1.40	1.40
TRACE-SUP-6	0.88	0.83	0.87
TRACE-NFD-4	**0.92**	**0.93**	**0.87**
Δ-stepping	0.84	0.88	0.82

G. Sensitivity to Inter-Domain Synchronization Constraints

This experiment evaluates the robustness of routing algorithms under constrained inter-domain synchronization in multi-domain scenarios. We compare TRACE-NFD-4 against GAT-SUP-6, GTN-SUP-6, and Δ-stepping in the C20–D30/D50/D100 multi-domain setting under random failures with $p_{fail}^{\max} = 0.05$. To emulate inter-domain dynamics, we abstract the synchronization rate as the available signaling budget for cross-domain state exchange. By limiting this rate, we simulate scenarios with incomplete or outdated inter-domain information, and evaluate routing robustness in terms of success rate degradation.

We model inter-domain synchronization as a lightweight summary exchange process. Each domain periodically reports a bounded set of egress summaries, where each summary entry contains key routing information (e.g., domain identifier, candidate egress node, and estimated routing cost). Let S denote the size of one summary entry. Let S denote the size of one summary entry. If each domain exchanges K summary entries for each destination domain every synchronization period T_s, the synchronization volume is approximated as (29)

$$V_{\text{sync}} \approx (M-1)KS, \tag{29}$$

where M is the number of domains. The corresponding synchronization rate is defined as (30)

$$R_{\text{sync}} = \frac{V_{\text{sync}}}{T_s}. \tag{30}$$

In the experiments, we vary R_{sync} by adjusting K and T_s to emulate different levels of inter-domain information availability.

Based on the above setup, we first quantify the synchronization sensitivity of different methods in Table VII. Specifically, R_{sync}^{95} denotes the minimum synchronization rate required to achieve a routing success rate of 95%. A lower value indicates that the method can maintain stable routing performance with more limited inter-domain information.

As shown in Table VII, Δ-stepping achieves the lowest synchronization requirement, with thresholds of 0.84, 0.88, and 0.82 in D30 +C20, D50 +C20, and D100 +C20, respectively. TRACE-SUP and TRACE-NFD achieve comparable thresholds, where TRACE-SUP requires 0.88, 0.83, and 0.87, while TRACE-NFD requires 0.92, 0.93, and 0.87. This indicates that TRACE can maintain routing reliability close to deterministic shortest-path search under constrained inter-domain synchronization. In contrast, GTN consistently requires a much higher synchronization rate of 1.40 across all three settings, and GAT shows larger variation from 0.93 to 1.03, indicating stronger sensitivity to incomplete inter-domain information.

Figure 12 (a)–(c) further illustrates the success-rate trends under different synchronization rates. As the synchronization rate increases, all methods generally achieve higher routing success rates because fresher and more complete inter-domain state information becomes available.

In the low-synchronization regime, where inter-domain information is severely limited, TRACE exhibits competitive or even slightly better performance compared to Δ-stepping in some cases, as shown in Figure 12 (a) and Figure 12 (b). This is because Δ-stepping relies more directly on accurate and up-to-date link-state information, while TRACE can leverage learned routing patterns to make more robust decisions under partial state visibility.

As the synchronization rate increases, Δ-stepping quickly approaches near-perfect routing success rates, reflecting its advantage in deterministic shortest-path computation when sufficient topology information is available. In contrast, learning-based methods, including TRACE, gradually converge to similar performance levels, demonstrating their ability to approximate optimal routing decisions given adequate inter-domain information.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

In larger-scale scenarios such as D100, as shown in Figure 12 (c), Δ-stepping achieves consistently higher success rates in the high-synchronization regime, due to its exact utilization of explicit link weights in a more complex topology. Meanwhile, TRACE maintains stable performance across different synchronization levels, without significant degradation under reduced synchronization, which is consistent with the relatively stable thresholds observed in Table VII.

Overall, these results indicate that while deterministic methods achieve optimal performance when full information is available, the proposed TRACE framework provides a more robust and scalable alternative under constrained inter-domain synchronization, achieving competitive performance with reduced sensitivity to incomplete topology information.

V. Conclusion

This work presented TRACE, a topology-aware Transformer architecture with adjacency-constrained encoding, together with NFD, a navigator–follower self-distillation mechanism designed for resilient and efficient routing in large LEO constellations. By coupling structure-aware attention with Monte Carlo–driven policy refinement, the proposed approach enables agents to learn stable routing behaviors under both random failures and targeted attacks. We further developed a hierarchical distributed routing architecture that extends these capabilities to multi-domain settings while maintaining controllable inter-domain errors.

Extensive simulations demonstrate that TRACE with NFD achieves near-optimal routing accuracy, significantly improves throughput, and offers strong robustness compared with mainstream routing algorithms. The results underscore the benefits of integrating topology-constrained neural architectures with distributed execution for scalable LEO routing, advancing the broader goal of efficient communication in future SAGINs, while supporting emerging security-aware applications such as radio frequency fingerprinting, device authentication, and anomaly detection.

References

[1] E. Lagunas, S. Chatzinotas, and B. Ottersten, "Low-earth orbit satellite constellations for global communication network connectivity," *Nature Rev. Electr. Eng.*, vol. 1, no. 10, pp. 656–665, Sep. 2024.

[2] Y. Li et al., "Stable hierarchical routing for operational LEO networks," in *Proc. 30th Annu. Int. Conf. Mobile Comput. Netw.*, May 2024, pp. 296–311.

[3] J. Zhang, B. Zhang, X. Cao, J. Wang, H. Dai, and S. Li, "Space information networks: Architecture, technologies, and research issues," *IEEE Wireless Commun.*, vol. 23, no. 2, pp. 98–105, Feb. 2016.

[4] G. Giambene, S. Kota, and P. Pillai, "NGSO satellite constellations: Communication and networking," *IEEE Commun. Mag.*, vol. 58, no. 4, pp. 21–27, Apr. 2020.

[5] O. Kodheli et al., "Satellite communications in the new space era: A survey and future challenges," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 1, pp. 70–109, 1st Quart., 2021.

[6] R. Radhakrishnan et al., "Survey of inter-satellite communication for small satellite systems: Physical layer to network layer view," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 4, pp. 2442–2473, 4th Quart., 2016.

[7] Y. Ran, Y. Ding, S. Chen, J. Lei, and J. Luo, "Fully-distributed dynamic packet routing for LEO satellite networks: A GNN-enhanced multi-agent reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 74, no. 3, pp. 5229–5234, Mar. 2025.

[8] O. Kodheli et al., "Satellite communications in the new space era: A survey and future challenges," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 1, pp. 70–109, Jan. 2021.

[9] X. Shi et al., "Graph processing on gpus: A survey," *ACM Comput. Surv.*, vol. 50, no. 6, pp. 1–35, Jan. 2018.

[10] C. Han, W. Xiong, and R. Yu, "Load-balancing routing for LEO satellite network with distributed hops-based back-pressure strategy," *Sensors*, vol. 23, no. 24, p. 9789, Dec. 2023.

[11] H. Dui, J. Zhai, and X. Fu, "Attack strategies and reliability analysis of wireless mesh networks considering cascading failures," *Rel. Eng. Syst. Saf.*, vol. 257, May 2025, Art. no. 110832.

[12] Z. Lai et al., "LeoCC: Making internet congestion control robust to LEO satellite dynamics," in *Proc. ACM SIGCOMM Conf.*, Sep. 2025, pp. 129–146.

[13] Z. Du, J. Jiao, H. Liu, Y. Wang, and Q. Zhang, "Multi-attribute consistency segment resilient routing for LEO satellite mega constellations," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10823–10839, Oct. 2025.

[14] W. Wu and X. Huang, "Split-LEO: Efficient AI model training over LEO satellite networks," *Sci. China Inf. Sci.*, vol. 68, no. 9, Sep. 2025, Art. no. 190305.

[15] X. Zhou et al., "Distributed routing and data scheduling in IPNs with GNN-based multiagent DRL," *IEEE Internet Things J.*, vol. 12, no. 12, pp. 21565–21576, Jun. 2025.

[16] Z. Zhao, G. Verma, C. Rao, A. Swami, and S. Segarra, "Distributed scheduling using graph neural networks," 2020, *arXiv:2011.09430*.

[17] C. Wang et al., "Bi-level multiobjective evolutionary learning: A case study on multitask graph neural topology search," *IEEE Trans. Evol. Comput.*, vol. 28, no. 1, pp. 208–222, Feb. 2024.

[18] F. Lozano-Cuadra, B. Soret, I. Leyva-Mayorga, and P. Popovski, "Continual deep reinforcement learning for decentralized satellite routing," *IEEE Trans. Commun.*, vol. 73, no. 10, pp. 8996–9012, Oct. 2025.

[19] S. Mahboob and L. Liu, "Revolutionizing future connectivity: A contemporary survey on AI-empowered satellite-based non-terrestrial networks in 6G," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 1279–1321, 2nd Quart., 2024.

[20] R. Zhang et al., "Generative AI agents with large language model for satellite networks via a mixture of experts transmission," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 12, pp. 3581–3596, Dec. 2024.

[21] R. Zhang et al., "Generative AI for space-air-ground integrated networks," *IEEE Wireless Commun.*, vol. 31, no. 6, pp. 10–20, Dec. 2024.

[22] J. Huang, Y. Yang, J. Lee, D. He, and Y. Li, "Deep reinforcement learning-based resource allocation for RSMA in LEO satellite-terrestrial networks," *IEEE Trans. Commun.*, vol. 72, no. 3, pp. 1341–1354, Mar. 2024.

[23] C. Lei et al., "Joint partitioning, allocation, and transmission optimization for federated learning in satellite constellations via multi-task MARL," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10345–10362, Oct. 2025.

[24] V. That, S. Muy, and J.-R. Lee, "Federated learning-based spectrum and energy efficiency enhancement in HAP-assisted LEO satellite communication," *Expert Syst. Appl.*, vol. 296, Jan. 2026, Art. no. 128916.

[25] M. Elmahallawy, T. Luo, and K. Ramadan, "Communication-efficient federated learning for LEO constellations integrated with HAPs using hybrid NOMA-OFDM," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 5, pp. 1097–1114, May 2024.

[26] Z. Zhai, Q. Wu, S. Yu, R. Li, F. Zhang, and X. Chen, "FedLEO: An offloading-assisted decentralized federated learning framework for low Earth orbit satellite networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5260–5279, May 2024.

[27] Z. Lin, Z. Chen, Z. Fang, X. Chen, X. Wang, and Y. Gao, "FedSN: A federated learning framework over heterogeneous LEO satellite networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 3, pp. 1293–1307, Mar. 2025.

[28] D.-J. Han, S. Hosseinalipour, D. J. Love, M. Chiang, and C. G. Brinton, "Cooperative federated learning over ground-to-satellite integrated networks: Joint local computation and data offloading," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 5, pp. 1080–1096, May 2024.

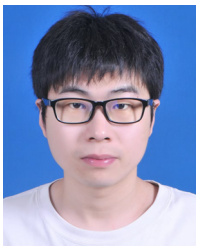
[29] Uditi and M. Arun, "Comparative analysis of parallelised shortest path algorithms using open MP," in *Proc. 3rd Int. Conf. Sens., Signal Process. Secur. (ICSSS)*, May 2017, pp. 369–378.

[30] N. Jasika, N. Alispahic, A. Elma, K. Ilvana, L. Elma, and N. Nosovic, "Dijkstra's shortest path algorithm serial and parallel execution performance analysis," in *Proc. 35th Int. Conv. MIPRO*, May 2012, pp. 1811–1815.

[31] G. Redinbo, "Queueing systems, volume I: Theory–leonard kleinrock," *IEEE Trans. Commun.*, vol. C-25, no. 1, pp. 178–179, Jan. 1977.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.

[32] D. Bertsekas and R. Gallager, *Data Networks*. Upper Saddle River, NJ, USA: Prentice-Hall, 1992.
[33] X. Wang, Y. Chen, and O. A. Dobre, "Designing robust 6G networks with bimodal distribution for decentralized federated learning," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, May 2024, pp. 1–6.
[34] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Represent.*, 2017. [Online]. Available: https://openreview.net/forum?id=SJU4ayYgl
[35] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. 6th Int. Conf. Learn. Represent.*, 2018. [Online]. Available: https://openreview.net/forum?id=rJXMpikCZ
[36] S. Yun, M. Jeong, R. Kim, J. Kang, and H. J. Kim, *Graph Transformer Networks*. Red Hook, NY, USA: Curran Associates Inc., 2019.
[37] K. Lei, P. Guo, Y. Wang, X. Wu, and W. Zhao, "Solve routing problems with a residual edge-graph attention neural network," *Neurocomputing*, vol. 508, pp. 79–98, Oct. 2022.
[38] A. M. Nagib, H. Abou-zeid, and H. S. Hassanein, "Toward safe and accelerated deep reinforcement learning for next-generation wireless networks," *IEEE Netw.*, vol. 37, no. 2, pp. 182–189, Mar. 2023, doi: 10.1109/MNET.106.2100578.
[39] Z. Xiong, Y. Zhang, D. Niyato, R. Deng, P. Wang, and L.-C. Wang, "Deep reinforcement learning for mobile 5G and beyond: Fundamentals, applications, and challenges," *IEEE Veh. Technol. Mag.*, vol. 14, no. 2, pp. 44–52, Jun. 2019.
[40] X. Cao et al., "Exploring LLM-based multi-agent situation awareness for zero-trust space-air-ground integrated network," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 6, pp. 2230–2247, Jun. 2025.
[41] C. Westphal, L. Han, and R. Li, "LEO satellite networking relaunched: Survey and current research challenges," 2023, *arXiv:2310.07646*.
[42] S. Kaur, "Analysis of inter-satellite free-space optical link performance considering different system parameters," *Opto-Electronics Rev.*, vol. 27, no. 1, pp. 10–13, Mar. 2019.
[43] U. Meyer and P. Sanders, "Delta-stepping: A parallelizable shortest path algorithm," *J. Algorithms*, vol. 49, no. 1, pp. 114–152, Oct. 2003.
[44] P. Prosser and C. Unsworth, "Limited discrepancy search revisited," *ACM J. Experim. Algorithmics*, vol. 16, Aug. 2011, Art. no. 1.6.
[45] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2018, *arxiv:1710.10903*.
[46] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.



Qiuchao Dai received the M.E. degree in computer technology from Xi'an Shiyou University, China, in 2024. He is currently pursuing the Ph.D. degree with the School of Cyber Engineering, Xidian University, Xi'an, China, focusing on distributed intelligent optimization and multi-agent collaborative techniques for next-generation LEO satellite networks.

Zhongyuan Jiang received the B.S. and Ph.D. degrees from Beijing Jiaotong University in 2009 and 2013, respectively. He is currently a Professor with the School of Cyber Engineering, Xidian University, China. His research interests include network function virtualization, mega-constellations, and next-generation wireless networks.

Fanxuan Sun received the B.E. degree in cyberspace security from Xidian University, Xi'an, China, in 2023, where he is currently pursuing the M.S. degree with the School of Cyber Engineering, focusing on distributed computing frameworks and LEO constellation communication.

Xinghua Li (Member, IEEE) received the M.S. and Ph.D. degrees in computer science from Xidian University, China, in 2004 and 2007, respectively. He is currently a Professor with the School of Cyber Engineering, Xidian University. His research interests include wireless network security, privacy protection, cloud computing, and security protocol formal methodology.

Jianfeng Ma (Member, IEEE) received the B.S. degree from Shaanxi Normal University in 1985 and the Ph.D. degree from Xidian University, China, in 1995. He is currently a Professor with the School of Cyber Engineering, Xidian University, and the Director of the Key Laboratory of Network and System Security of Shaanxi Province. His research interests include cryptography, wireless network security, data security, and mobile intelligent system security.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on September 03,2026 at 19:52:11 UTC from IEEE Xplore. Restrictions apply.